UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Computación Paralela y Distribuída

Proyecto - Laboratorio
Pregrado
2026-II
Profesor José Fiestas

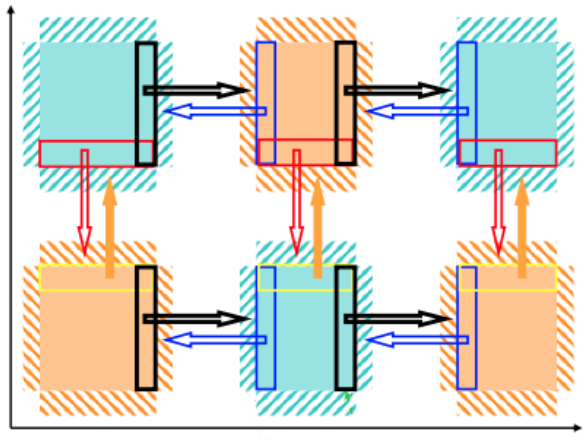
Indicaciones generales:

- Escoja **uno de los 9** proyectos mostrados
- El proyecto se desarrollará en grupos de 2 o 3 estudiantes. Los **nombres** de los integrantes deberán aparecer en el **Informe de Proyecto**
- Los archivos de proyecto se subirán directamente a **Canvas**
- La entrega parcial de proyecto será hasta el **domingo 27 de setiembre** , y la entrega final hasta el **domingo 22 de noviembre**, a medianoche.
- La presentación oral será
  - Parcial (avance): **lunes 28.09** y **martes 29.09**
  - Final: **lunes 23.11**, **martes 24.11** y **miércoles 25.11**
- La sección 8 describe **Indicaciones importantes para el proyecto**, y la sección 9, la **Rúbrica**

---

1 Ecuación de calor en 2D

En este proyecto debe paralelizar con MPI el código secuencial (adjunto)[1] que resuelve la ecuación de calor (EDP vista en clase).



El desarrollo y **análisis de resultados** debe contener lo siguiente:

a) Escoja el paradigma adecuado, elabore un PRAM y desarrolle un código en paralelo en C++

b) Registre el desarrollo del código en por lo menos 3 pasos (versiones beta). En caso utilice una fuente externa, mostrar la validación del código. Utilice descomposición del dominio considerando que n puede no ser divisible entre p. No todos los procesos tienen la misma cantidad de elementos. Utilice comunicación no-bloqueada y tipos derivados MPI

c) Realice análisis de tiempos y compare los resultados con la complejidad teórica esperada. Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (debe medir los FLOPs en el código usado).

d) Analice la escalabilidad del algoritmo con un tamaño variable del problema

Bibliografía

[1] heat.cpp

2 TSP

Resuelva el problema del agente viajero en paralelo utilizando el método *branch and bound* discutido en clase. Si utiliza otro método debe sustentarlo teóricamente.
Se trata de un vendedor que debe minimizar el recorrido entre n ciudades. Puede empezar en cualquiera de ellas y terminar en la misma luego del recorrido. Cada ciudad debe ser visitada solo una vez.

TSP se aplica a diversos problemas como DNA y genomas, conexiones de electricidad entre ciudades, conexión satelital, conexiones en fibra óptica, VLSI (Very Large System Integration)

Escoja una de las siguientes aplicaciones del algoritmo en paralelo:

- Aplique el algoritmo TSP desarrollado a un set VLSI del siguiente enlace: VLSI datasets. Escoja el set adecuado para que los resultados obtenidos sean claros. En caso los recursos disponibles (hardware) no sean suficientes para completar algunos de los casos VLSI, puede reducir la data original
- Utilice 10 puntos geográficos en sendos países del mundo y calcule las distancias entre ellos. Encuentre la ruta (aérea) más corta al recorrer los 10 puntos. Compare los resultados escogiendo ahora 20 puntos.

Puede escribir el código desde cero o utilizar algún código fuente para solucionar el problema, pero necesita validarlo, mostrando su **precisión** antes de usarlo para el caso planteado.

El desarrollo y **análisis de resultados** debe contener lo siguiente:

a) Escoja el paradigma adecuado, elabore un PRAM y desarrolle un código en paralelo en C++

b) Registre el desarrollo del código en por lo menos 3 pasos (versiones beta). En caso utilice una fuente externa, mostrar la validación del código. Si utiliza un método distinto al indicado, debe validarlo teóricamente

c) Realice análisis de tiempos y compare los resultados con la complejidad teórica esperada. Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (debe medir los FLOPs en el código usado).

d) Analice la escalabilidad del algoritmo con un tamaño variable del problema

3 Expresiones regulares

Expresiones regulares son una forma de representar un conjunto de cadenas de caracteres que satisfacen ciertas reglas de construcción (gramáticas). E.g. dado el alfabeto $\Sigma = \{0, 1\}$, la expresión regular $01(01)^*$ acepta cadenas como {01,0101,010101,...}. Expresiones regulares se usan frecuentemente en algoritmos de manipulación de strings, de análisis lexicográfico y sintáctico, entre otros. Un ejemplo de ello es NetKat, un lenguaje de programación de redes [1]

Un software de reconocimiento de expresiones regulares es de mucha utilidad, especialmente si es escalable. PAREM [2] es un ejemplo de un sotware de expresiones regulares rápido y escalable, con un speedup lineal con respecto al número de procesos. Este algoritmo particiona la cadena (input), para luego combinar los resultados parciales obtenidos (ver paper).

El desarrollo y **análisis de resultados** debe contener lo siguiente:

a) Escoja el paradigma adecuado, elabore un PRAM y desarrolle un código en paralelo en C++

b) Registro del desarrollo del código en por lo menos 3 pasos (versiones beta)

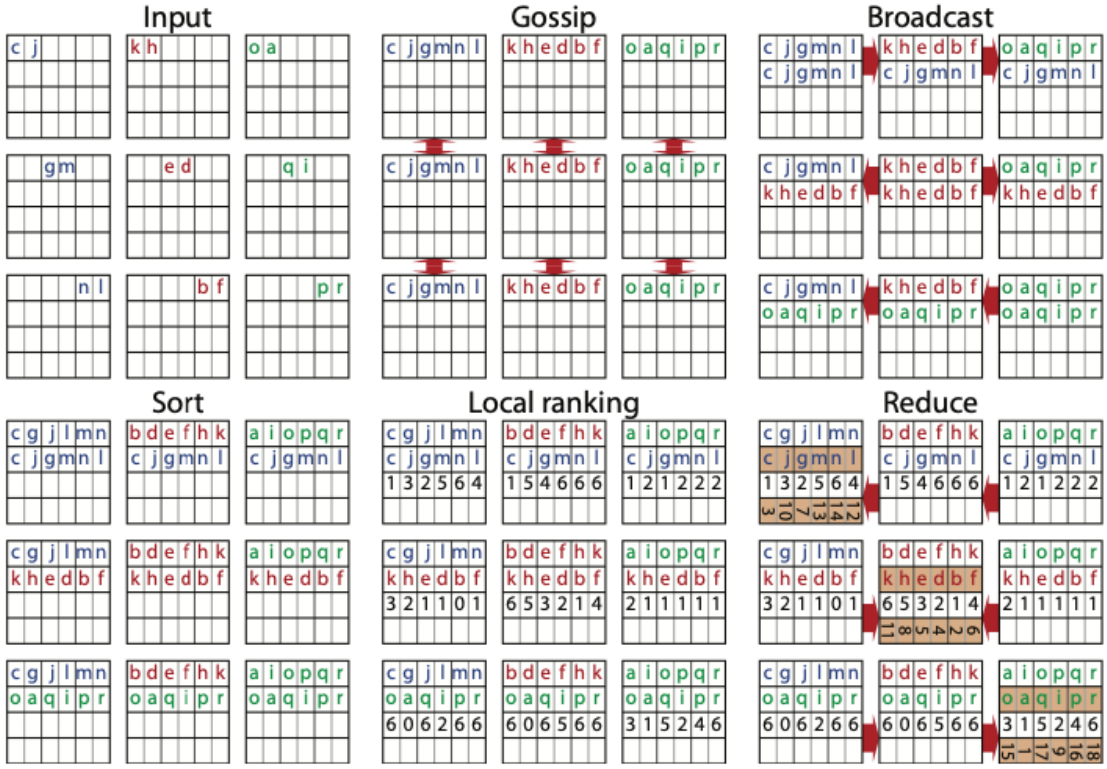
c) Realice el análisis de tiempos, comparando mediciones con la complejidad teórica del código. Para distinto número de procesos y tamaño del problema. Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (debe medir los FLOPs en el código usado).

d) Analice la escalabilidad del algoritmo con un tamaño variable del problema

Bibliografía:
[1] arxiv.org/pdf/1412.1741.pdf

4 Ordenamiento eficiente por ranking

Se pide desarrollar un código en paralelo para el algoritmo de ordenamiento por ranking visto en clase



El desarrollo y **análisis de resultados** debe contener lo siguiente:

a) Elaborar un PRAM y el código en paralelo (MPI)

b) Registro del desarrollo del código en por lo menos 3 pasos (versiones beta).

c) Mida los tiempos de **ejecución**, **cómputo** y **comunicación**, así como la velocidad del algoritmo, y representarlas en gráficas. Compare los resultados con la complejidad teórica en paralelo, para distinto número de procesos y tamaño del problema. Utilice un tamaño adecuado del array para lograr tiempos de ejecución medibles

d) Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (debe medir los FLOPs en el código usado).

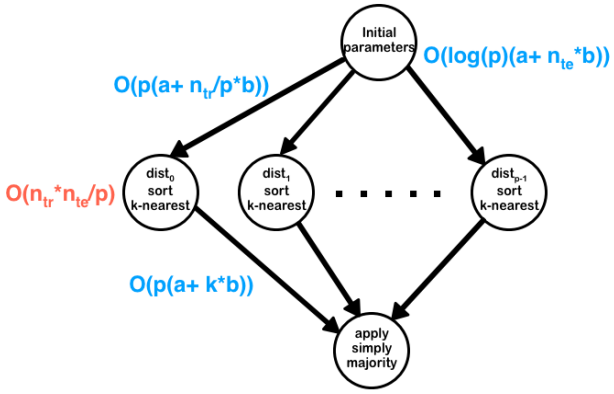
e) Compare la escalabilidad del software con Quicksort en paralelo, con un tamaño variable del problema

Bibliografía:

[1] Algoritmo similar, alternativo, en: Curso Algoritmos Paralelos, Peter Sanders, 2020/2021. Puede usarlo en vez del discutido en clase.

5 KNN

KNN es un algoritmo de clasificación supervisada (aunque también puede aplicarse a regresión) que clasifica un punto nuevo según las etiquetas de sus k vecinos más cercanos en el espacio de características. En paralelo, cada elemento de testeo puede ser calculado independientemente del resto con la fracción de la data de prueba que tiene cada proceso.



Paralelice el código adjunto *knn_digits_sec.py* [1], que utiliza la conocida librería *load_digits* para clasificar dígitos.
El desarrollo y **análisis de resultados** debe contener lo siguiente:

a) Elabore un PRAM y el código en Python incluyendo las directivas de comunicación (comm.bcast, comm.scatter, comm.gather)[2] de acuerdo al criterio mostrado en el DAG

b) Registre el desarrollo del código en por lo menos 3 pasos (versiones beta). En caso utilice una fuente externa, mostrar la validación del código

c) Mida los tiempos de **ejecución**, **cómputo** y **comunicación** (represéntelas gráficamente), así como la **precisión** del modelo (accuracy). Incremente tanto procesos (p) como datos (n). Utilice la complejidad teórica para **validar** el resultado obtenido. Describa como normaliza la expresión teórica para que pueda aproximar los experimentos. ¿Cuál estima, sería la cantidad **óptima de procesos**?

d) Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (FLOP por segundo). Obtenga los FLOP de la región paralelizable (entrenamiento), basada en la fórmula de la **distancia euclidiana entre dos puntos**.

e) Analice la escalabilidad del algoritmo con un tamaño variable del problema

Bibliografía:
[1] https://drive.google.com/file/d/1wTBSh7bwT1QVvce0pFMv5tcCWsg4-HeK/view?usp=sharing
[2] https://mpi4py.readthedocs.io/en/stable/

6 Marching Cubes

Marching Cubes es un algoritmo que recorre una malla 3D de valores escalares para extraer contornos/una superficie (malla triangular) correspondiente a un nivel de isovalor (isolevel). Se aplica e.g. en mapas topográficos, simulación de fluidos, tomografías médicas, reconstrucción (3D).

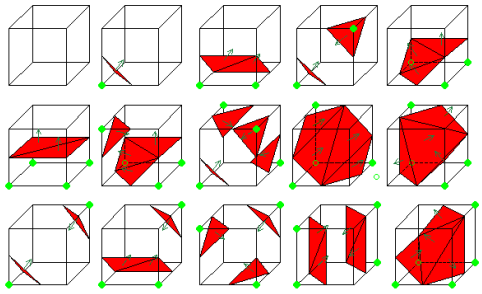


Figure 1: Configuración de Marching cubes

Implemente una versión paralela del algoritmo de Marching Cubes para acelerar la generación de contornos/iso-superficies a partir de campos escalares volumétricos
El desarrollo y **análisis de resultados** debe contener lo siguiente:

a) Escoja el paradigma adecuado, elabore un PRAM y desarrolle un código paralelo en C++. Registre el desarrollo del código en por lo menos 3 pasos (versiones beta)

b) Realice el análisis de tiempos, validando mediciones con la complejidad teórica del código, para distinto número de procesos y tamaño del problema. Utilice por lo menos matrices de 512 celdas por lado

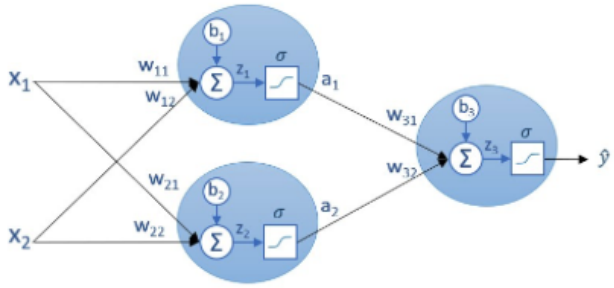
c) Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (debe medir los FLOPs en el código usado).

d) Analice la escalabilidad del algoritmo

Bibliografía:
[2] https://en.wikipedia.org/wiki/Marching˙cubes

7 Redes Neuronales

La multiplicación de matrices es fundamental en redes neuronales, ya que es la operación matemática central que permite:
1. Propagación hacia adelante (Forward Propagation) En cada capa de una red neuronal, los datos de entrada se transforman mediante una multiplicación de matriz con los pesos de esa capa: y=W X+b donde, X: entrada (vector o matriz), W: matriz de pesos, b: sesgo (bias), y: salida lineal
2. Entrenamiento y Backpropagation Durante el entrenamiento, el algoritmo de retropropagación también utiliza multiplicación de matrices para calcular los gradientes: $\delta = (W^T) \cdot \delta_{siguiente}$



$$W = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \end{bmatrix}$$

$T_s(n) = O(E \cdot n \cdot d \cdot h)$
E: número de épocas, **n:** número de muestras, **d:** número de features (entradas), **h:** número de neuronas por capa.

1) Desarrolle un PRAM y código en Python basado en el pseudocódigo secuencial mostrado.

2) Utilice distinto número de hilos (p=1,2,4,8,16, 32), y número de datos de muestra, dado por el parámetro n_samples (5000, 10000, 20000, 40000) y genere las gráficas de tiempos de ejecución correspondientes.

3) Registro del desarrollo del código en por lo menos 3 pasos (versiones beta)

4) Compare gráficamente los resultados con la expresión **teórica** de complejidad en paralelo. Debe superponer la curva teorica a la experimental encontrando los factores de proporcionalidad que conviertan la expresión bigO en tiempos de ejecución en segundos

5) Calcule el **speedup** y **eficiencia** y haga un análisis de **escalabilidad** basado en los resultados obtenidos. Determine experimentalmente, si E se mantiene constante

Algorithm 1: Entrenamiento secuencial de múltiples MLP con distintas semillas

Data: Datos (X, y), semillas $S = \{0, 1, \ldots, p-1\}$, hiperparámetros $(\alpha, \eta, h, T_{\max})$
Result: Modelo M^* con mayor exactitud
`/* División única de datos */`
Dividir (X, y) en (X_{train}, y_{train}) y (X_{test}, y_{test});
`/* Inicialización */`
$A_{best} \leftarrow -\infty$;
$s^* \leftarrow -1$;
$M^* \leftarrow \varnothing$;
`/* Entrenamiento secuencial de múltiples MLP */`
foreach $s \in S$ **do**
    Inicializar generador aleatorio con semilla s;
    Construir modelo M_s con parámetros $(\alpha, \eta, h, T_{\max})$;
    Entrenar M_s con (X_{train}, y_{train});
    Calcular exactitud:

$$A_s \leftarrow \text{accuracy}(M_s, X_{test}, y_{test})$$

    if $A_s > A_{best}$ **then**
        $A_{best} \leftarrow A_s$;
        $s^* \leftarrow s$;
        $M^* \leftarrow M_s$;
    end
end
Guardar M^* en archivo `mlp_model.joblib`;
return M^*

8 Evolución galáctica

Simulaciones de evolución galáctica permiten predecir teóricamente el comportamiento dinámico de estrellas en galaxias y cúmulos globulares. El modelo requiere conjuntos de objetos (N



estrellas) de una cantidad muy grande, billones de estrellas en la realidad, que exigen una simulación lo más realistica posible, pero cuyos resultados sean accesibles en un tiempo mesurado.

Esta propuesta se basa en el trabajo hecho por el grupo Astrofísica de la Universidad de Heidelberg, Alemania. Para ello, se cuenta con el código hibrido de NCuerpos PhiGPU [1].

El desarrollo y **análisis de resultados** debe contener lo siguiente:

a) Elabore un PRAM del algoritmo basandose en el código de NCuerpos adjunto y determine la complejidad teórica del mismo

b) Ejecute mediciones de tiempo con el código y compárelas con la teoría, para distinto número de procesos y tamaño del problema. El código cuenta con un Makefile que se ejecuta con: *make cpu-4th* Para modificar el tamaño del problema (cantidad de elementos) utilice en código *gen-plum.c* en la misma carpeta, modificando la variable N (e.g. $10^3 - 10^4$)

d) Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (debe medir los FLOPs en el código usado).

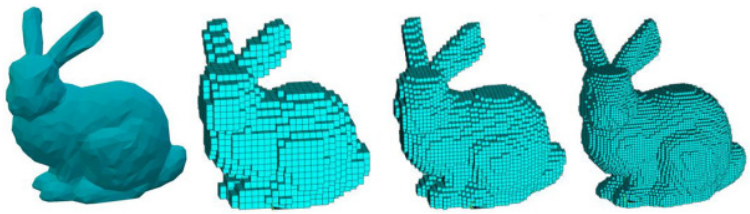
e) Analice la escalabilidad del algoritmo con un tamaño variable del problema

Bibliografía

[1] phiGPU code
[2] Paper2011

9 Voxelization

Las representaciones binarias de vóxeles (unidad mínima de un espacio 3D discreto) se emplean ampliamente en gráficos por computadora. Generalmente se obtienen como aproximaciones discretas de objetos mediante un proceso llamado voxelización.



(a) Bunny model (b) 1 mm (c) 0.6 mm (d) 0.4 mm

Las voxelizaciones se utilizan en dominios tan diversos como el emparejamiento de formas 3D y el procesamiento de visibilidad. Dado que la voxelización suele ser una parte integral crítica en términos de rendimiento, se han propuesto varios algoritmos para realizar la voxelización de mallas triangulares en la GPU, aprovechando la rápida tubería de rasterización para esta conversión de escaneo 3D.

El desarrollo y **análisis de resultados** debe contener lo siguiente:

a) Escoja el paradigma adecuado, elabore un PRAM y desarrolle un código paralelo en C++

b) Registro del desarrollo del código en por lo menos 3 pasos (versiones beta)

c) Realice el análisis de tiempos, comparando mediciones con la complejidad teórica del código. Para distinto número de procesos y tamaño del problema.

d) Derive la gráfica de **speedup** de los datos anteriores y genere una gráfica de **FLOPs vs. p** (debe medir los FLOPs en el código usado).

e) Analice la escalabilidad del algoritmo

10 Indicaciones importantes para la entrega del proyecto

10.1 Informe parcial de proyecto

Se pide realizar un informe del avance del proyecto, que contenga la siguiente información:

- **Nombre del proyecto**, y de los **integrantes**
- **Porcentaje de participación** de cada integrante en el proyecto (de 0 a 100 %, 100 es que ha contribuido en forma equitativa al resto)
- **Introducción:** Describa el proyecto elegido y la solución planteada
- **Método (punto 1 de la rúbrica):** Describa el algoritmo usado (PRAM). Describa las métricas solicitadas (Tiempo de ejecución, Speedup, Eficiencia) tal que sea posible leerlas del pseudocógido.
- **Bibliografía (punto 3 de la rúbrica):** Describa las fuentes utilizadas y referenciadas en el documento (e.g. IA). Describa su impacto en el desarrollo del proyecto.

10.2 Rúbrica (entrega parcial)

Se utilizará la siguiente rúbrica para la calificación del avance de proyecto

Criterio	Logrado	Parcialmente Logrado	No Logrado
1. Desarrollo de PRAM y métricas	El diseño del algoritmo es adecuado. Las métricas corresponden al PRAM (`6 pts`)	Existen algunos errores menores en el diseño del algoritmo y determinación de las métricas (`3 pts`)	Existen muchos errores en el diseño del algoritmo y métricas (`1pt`).
2. Uso correcto de fuentes externas	Justifica el uso de fuentes externas (publicaciones, charlas, IA) (`4 pts`)	Justifica solo parcialmente el uso de fuentes externas (`2 pts`).	No justifica el uso de fuentes externas (`1pt`).
3. Presentación escrita (informe) y trabajo en grupo **	Describe los puntos del informe en forma ordenada y satisfactoria (`4 pts`)	No describe en forma adecuada todos los puntos del informe (`2 pts`).	No describe en forma adecuada ningún punto del informe (`1pt`).
4. Presentación oral	Muestra dominio del contenido del proyecto durante presentación (`6 pts`)	No se demostró un dominio total del contenido del proyecto durante la presentación (`3 pts`).	No se presentó conocimiento del proyecto durante la presentación (`1pt`).

: se aplicará una **penalidad de 2 puntos si no se desarrolla el proyecto en formal grupal

10.3 Informe final de proyecto

Se pide realizar un informe de los resultados del proyecto, que contenga la siguiente información:

- **Nombre del proyecto**, y de los **integrantes**
- **Porcentaje de participación** de cada integrante en el proyecto (de 0 a 100 %, 100 es que ha contribuido en forma equitativa al resto)
- **Introducción:** Describa el proyecto elegido y la solución planteada
- **Método (punto 1 de la rúbrica):** Describa el algoritmo usado (PRAM) y/o el código.
- **Resultados y análisis (punto 2 de la rúbrica):** Describa y analice los resultados obtenidos y plantee mejoras a la solución.
- **Bibliografía (punto 3 de la rúbrica):** Describa las fuentes utilizadas y referenciadas en el documento (e.g. IA). Describa su impacto en el desarrollo del proyecto.

10.4 Rúbrica (entrega final)

Se utilizará la siguiente rúbrica para la calificación de la entrega final del proyecto

Criterio	Logrado	Parcialmente Logrado	No Logrado
1. Desarrollo de código/PRAM	El diseño del algoritmo y/o código es adecuado (`2 pts`)	Existen algunos errores menores en el diseño del algoritmo o funcionamiento del código (`1 pt`)	Existen muchos errores en el diseño del algoritmo y/o funcionamiento del código (`0.5pts`).
2. Resultados y Análisis	Desarrolla correctamente las preguntas de análisis de resultados (`4 pts`)	Desarrolla parcialmente las preguntas de análisis de resultados (`2 pts`).	Desarrolla deficientemente las preguntas de análisis de resultados (`1pt`).
3. Uso correcto de fuentes externas	Justifica el uso de fuentes externas (publicaciones, charlas, IA) (`4 pts`)	Justifica solo parcialmente el uso de fuentes externas (`2 pts`).	No justifica el uso de fuentes externas (`1pts`).
4. Presentación escrita (informe) y trabajo en grupo **	Describe los puntos del informe en forma ordenada y satisfactoria (`4 pts`)	No describe en forma adecuada todos los puntos del informe (`2 pts`).	No describe en forma adecuada ningún punto del informe (`1pt`).
5. Presentación oral	Muestra dominio del contenido del proyecto durante presentación (`6 pts`)	No se demostró un dominio total del contenido del proyecto durante la presentación (`2 pts`).	No se presentó conocimiento del proyecto durante la presentación (`1pt`).

: se aplicará una **penalidad de 2 puntos si no se desarrolla el proyecto en formal grupal

10.5 Entrega grupal

El proyecto se realizará en forma **grupal (2 o 3 integrantes)**
El proyecto se subirá a Canvas (Proyecto Laboratorio)
Presente el documento de proyecto en formato **.pdf**. Puede convertir un documento a pdf o usar una plantilla de Plantillas Latex . Asimismo, adjunte el **código de programación** o **link a Github**.